

# 웹 취약점 진단 보고서

Web Application Vulnerability Assessment Report

## 1. 진단 사이트 개요

|          |                                    |
|----------|------------------------------------|
| 진단 호스트   | localhost / Target Server (Masked) |
| 종합 진단 결과 | 취약 발견 (VULNERABLE)                 |

본 보고서는 마스킹 처리된 패킷 데이터를 기반으로 LLM이 진단한 결과 및 판정 근거를 포함합니다.

## 2. 진단 결과 요약 및 상세 분석

|                        |                                                                                                                                                                      |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 취약점 명칭                 | SQL Injection                                                                                                                                                        |
| OWASP 분류               | A05 Injection                                                                                                                                                        |
| 위험도<br>(Severity)      | HIGH                                                                                                                                                                 |
| 판단 근거 요약               | 요청에서 사용자가 제어할 수 있는 'searchstring' 파라미터에 SQL 주입 공격 패턴이 포함되어 있으며, 응답에서 SQL 주입 공격이 성공했음을 나타내는 오류 메시지나 비정상적인 동작이 확인되지 않았지만, 검색 결과가 정상적으로 반환된 점에서 SQL 주입이 성공했을 가능성이 높습니다. |
| 요청 분석 근거<br>(Request)  | 쿼리 파라미터 'searchstring'에 SQL 주입 패턴인 "') order by 1#"가 포함되어 있습니다.                                                                                                      |
| 응답 분석 근거<br>(Response) | 응답 본문에 검색 결과가 정상적으로 반환되었으며, 이는 SQL 주입이 성공했을 가능성을 시사합니다.                                                                                                              |
| 보안 영향<br>(Impact)      | SQL 주입 공격이 성공할 경우 데이터베이스의 민감한 데이터에 접근하거나 데이터베이스의 구조를 변경할 수 있습니다.                                                                                                     |

### 3. 프레임워크별 시큐어코딩 및 패치 가이드

**대응 방안 요약:** 사용자 입력을 안전하게 처리하기 위해 준비된 문(statement)이나 ORM을 사용하여 SQL 쿼리를 작성하십시오.

#### Spring (Java)

##### 취약한 코드 (Vulnerable)

```
String query = "SELECT * FROM users WHERE user_id = " + userId + "";  
Statement stmt = connection.createStatement();
```

##### 안전한 패치 코드 (Secure)

```
String query = "SELECT * FROM users WHERE user_id = ?";  
PreparedStatement pstmt = connection.prepareStatement(query);  
pstmt.setString(1, userId);
```

#### Flask (Python)

##### 취약한 코드 (Vulnerable)

```
query = f"SELECT * FROM users WHERE user_id = '{user_id}'"  
cursor.execute(query)
```

##### 안전한 패치 코드 (Secure)

```
query = "SELECT * FROM users WHERE user_id = %s"  
cursor.execute(query, (user_id,))
```

#### Node.js

##### 취약한 코드 (Vulnerable)

```
const query = `SELECT * FROM users WHERE user_id = '${userId}'`;  
db.query(query);
```

##### 안전한 패치 코드 (Secure)

```
const query = 'SELECT * FROM users WHERE user_id = ?';  
db.query(query, [userId], callback);
```